

## CODE:

```
1      [org 0x100]
2
3      jmp start
4
5      multiplicand: db 13
6
7      multiplier:   db 5
8
9      result:       db 0
10
11     start:         mov cl, 4
12                   mov bl, [multiplicand]
13                   mov dl, [multiplier]
14
15     checkbit:      shr dl, 1
16                   jnc skip
17                   add [result], bl
18
19     skip:          shl bl, 1
20                   dec cl
21                   jnz checkbit
22
23                   mov ax, 0x4c00
24                   int 0x21
```

DOSBox 0.74-3,...

AX:0000 SI:0000 CS:19F5 IP:0110 Stack:0000 Flags:7200  
BX:000D DI:0000 DS:19F5 +2:20CD  
CX:0004 BP:0000 ES:19F5 +4:9FFF OF:DF IF:SF ZF:AF PF:C  
DX:0005 SP:FFFE SS:19F5 FS:19F5 +6:EA00 0 0 1 0 0 0 0

CMD > █

010C 8A160401 MOV DL,0101  
0110 D0EA SHR DL,1  
0112 7304 JNC 0118  
0114 001E0501 ADD [0105],BL  
0118 D0E3 SHL BL,1  
011A FEC9 DEC CL  
011C 75F2 JNZ 0110  
011E B8004C MOV AX,4C00  
0121 CD21 INT 21

DS:0100 E9 03 00 00 05 00 B1 0  
DS:0108 8A 1E 03 01 8A 16 04 0  
DS:0110 D0 EA 73 04 00 1E 05 0  
DS:0118 D0 E3 FE C9 75 F2 B8 0  
DS:0120 4C CD 21 8B 46 F6 D1 E  
DS:0128 D1 E0 C5 5E D8 01 C3 8  
DS:0130 07 8B 57 02 85 D2 75 0  
DS:0138 85 C0 74 1C C7 46 DC 0  
DS:0140 00 0E 5E FC 03 7D 0E 0  
DS:0148 74 09 8B 46 F2 48 3B 4

2 0 1 2 3 4 5 6 7 8 9 A B C D E F = f.8= i.+...  
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 .....R. ....  
DS:0010 1B 01 10 01 18 01 92 01 01 01 01 00 FF 00 01 00 .....  
DS:0020 01 FF FF FF FF FF FF FF FF FF FF EB 19 C0 11 .....  
DS:0030 A2 01 14 00 18 00 F5 19 FF FF FF FF 00 00 00 00 .....J. ....  
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....  
Step 2ProcStep 3Retrieve 4Help ON 5BRK Menu 6 7 up 8 dn 9 le 10 ri

Why shifting right?

To check if DX(multiplier) has 0 or 1 in its

LSB :

- If its 0 no operation to be performed only shifting required

- If its 1 add multiplier to the result and then do the shift

Before shifting :

- BX stores 000D (1101 in binary).
- CX stores the no of iteration required for multiplication(max number of bits)
- DX stores the multiplier 5 (0101).
- The memory location result (0105) stores value of result which is 0 initially (this will change when we add into result)

After the first shift:

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 0112 Stack +0 0000 Flags 7211
BX 0000 DI 0000 DS 19F5 +2 20CD
CX 0004 BP 0000 ES 19F5 HS 19F5 +4 9FFF
DX 0002 SP FFFE SS 19F5 FS 19F5 +6 EA00

CMD >
0110 D0EA SHR DL,1
0112 7304 JNC 0118
0114 001E0501 ADD [0105],BL
0118 D0E3 SHL BL,1
011A FEC9 DEC CL
011C 75F2 JNZ 0110
011E B8004C MOV AX,4C00
0121 CD21 INT Z1
0123 B846F6 MOV AX,[BP-0A]
  
```

- DX has changed from : **0101(5)**->**0010(2)** {%2}
- Least significant bit before rotation **sent to CF**

Now as CX flag has been raised the result 0010(2) will be added to the result so it would execute the next instruction(ADD) and not jump:

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 0110 Stack +0 0000 Flags 7200
BX 0000 DI 0000 DS 19F5 +2 20CD
CX 0004 BP 0000 ES 19F5 HS 19F5 +4 9FFF
DX 0002 SP FFFE SS 19F5 FS 19F5 +6 EA00

CMD >
0114 001E0501 ADD [0105],BL
0118 D0E3 SHL BL,1
011A FEC9 DEC CL
011C 75F2 JNZ 0110
011E B8004C MOV AX,4C00
0121 CD21 INT Z1
0123 B846F6 MOV AX,[BP-0A]
0126 D1D0 SHL AX,1
0128 D1E0 SHL AX,1
  
```

The memory location 0105 now stores 0D:

- Value of multiplicand has been added into result as CF was raised

SHL will shift 1 bit left the value of bx which changes from **0000 1101(13)**->**0001 1010(A)**:

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 011A Stack +0 0000 Flags 7210
BX 001A DI 0000 DS 19F5 +2 20CD
CX 0004 BP 0000 ES 19F5 HS 19F5 +4 9FFF
DX 0002 SP FFFE SS 19F5 FS 19F5 +6 EA00

CMD >
0118 D0E3 SHL BL,1
011A FEC9 DEC CL
011C 75F2 JNZ 0110
011E B8004C MOV AX,4C00
0121 CD21 INT Z1
0123 B846F6 MOV AX,[BP-0A]
0126 D1D0 SHL AX,1
0128 D1E0 SHL AX,1
012A C5ED8 LDS BX,[BP-20]
  
```

After this step cx will be dec by 1 as 1 iteration has been completed and result is added:

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 011A Stack +0 0000 Flags 7210
BX 001A DI 0000 DS 19F5 +2 20CD
CX 0001 BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 0002 SP FFEE SS 19F5 FS 19F5 +6 EA00 0 0 1 0 0 1 0 0

CMD >
011A DEC3 SHL BL,1
011A FEC9 DEC CL
011C 75F2 JNZ 0110
011E B804C MOV AX,4C00
0121 CD21 INT 21
0123 B846F6 MOV AX,[BP-0A]
0126 D1E9 SHL AX,1
012B D1E9 SHL AX,1
012A C55ED8 LDS BX,[BP-2B]
012D 01C3 ADD BX,AX

2
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.0= i.l.+...
DS:0010 18 01 10 01 18 01 92 01 01 01 01 00 FF 00 01 00 .....f. ....
DS:0020 01 00 01 FF FF FF FF FF FF FF FF FF EB 19 C0 11 ... ..L.
DS:0030 A2 01 14 00 18 00 F5 19 FF FF FF FF 00 00 00 00 0. ....J.
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 011C Stack +0 0000 Flags 7204
BX 001A DI 0000 DS 19F5 +2 20CD
CX 0000 BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 0002 SP FFEE SS 19F5 FS 19F5 +6 EA00 0 0 1 0 0 1 0 0

CMD >
011A DEC3 SHL BL,1
011A FEC9 DEC CL
011C 75F2 JNZ 0110
011E B804C MOV AX,4C00
0121 CD21 INT 21
0123 B846F6 MOV AX,[BP-0A]
0126 D1E9 SHL AX,1
012B D1E9 SHL AX,1
012A C55ED8 LDS BX,[BP-2B]
012D 01C3 ADD BX,AX

2
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.0= i.l.+...
DS:0010 18 01 10 01 18 01 92 01 01 01 01 00 FF 00 01 00 .....f. ....
DS:0020 01 00 01 FF FF FF FF FF FF FF FF FF EB 19 C0 11 ... ..L.
DS:0030 A2 01 14 00 18 00 F5 19 FF FF FF FF 00 00 00 00 0. ....J.
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

Now jnz will check if whole bits of multiplier have been processed or not which is indicated by 0 flag :

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 011C Stack +0 0000 Flags 7204
BX 001A DI 0000 DS 19F5 +2 20CD
CX 0003 BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 0002 SP FFEE SS 19F5 FS 19F5 +6 EA00 0 0 1 0 0 1 0 0

CMD >
011A DEC3 SHL BL,1
011A FEC9 DEC CL
011C 75F2 JNZ 0110
011E B804C MOV AX,4C00
0121 CD21 INT 21
0123 B846F6 MOV AX,[BP-0A]
0126 D1E9 SHL AX,1
012B D1E9 SHL AX,1
012A C55ED8 LDS BX,[BP-2B]
012D 01C3 ADD BX,AX

2
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.0= i.l.+...
DS:0010 18 01 10 01 18 01 92 01 01 01 01 00 FF 00 01 00 .....f. ....
DS:0020 01 00 01 FF FF FF FF FF FF FF FF FF EB 19 C0 11 ... ..L.
DS:0030 A2 01 14 00 18 00 F5 19 FF FF FF FF 00 00 00 00 0. ....J.
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

As zero flag is not raised it means that some iteration is Left so do the whole process again until ZF is raised

2<sup>ND</sup> ITERATION: NOW WE WILL PROCESS SECOND MSB OF MULTIPLIER 0101:

```

DOSBox 0.74-3,...
AX 0000 SI 0000 CS 19F5 IP 0112 Stack +0 0000 Flags 7210
BX 001A DI 0000 DS 19F5 +2 20CD
CX 0003 BP 0000 ES 19F5 HS 19F5 +4 9FFF OF DF IF SF ZF AF PF CF
DX 0001 SP FFEE SS 19F5 FS 19F5 +6 EA00 0 0 1 0 0 1 0 0

CMD >
0110 D0EA SHR DL,1
0112 7304 JNC 0118
0114 00E0501 ADD [0051],BL
0118 DEC3 SHL BL,1
011A FEC9 DEC CL
011C 75F2 JNZ 0110
011E B804C MOV AX,4C00
0121 CD21 INT 21
0123 B846F6 MOV AX,[BP-0A]

2
DS:0000 CD 20 FF 9F 00 EA F0 FE AD DE 1B 05 C5 06 00 00 = f.0= i.l.+...
DS:0010 18 01 10 01 18 01 92 01 01 01 01 00 FF 00 01 00 .....f. ....
DS:0020 01 00 01 FF FF FF FF FF FF FF FF FF EB 19 C0 11 ... ..L.
DS:0030 A2 01 14 00 18 00 F5 19 FF FF FF FF 00 00 00 00 0. ....J.
DS:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

Shift right operation has been performed which changes DX from 0010(2)->0001(1) %2.

And 2<sup>nd</sup> MSB of 5 now MSB will be sent to CF so CF Remain 1.

Now we will check if 2<sup>nd</sup> MSB is 0 or 1 as 2<sup>nd</sup> MSB of 5(0101) is 0 carry would not be raised because of which the step of adding bits into result would be skipped and program directly would jump to shift left:

From above we see no addition command has been executed.

And the memory location 0105 still stores the same result as before (0D)

Similarly next 2 iterations are performed so as it is already demonstrated I will now skip to show how program ends:

When cx becomes 0 :

- 0 flag is raised
- Due to which no jump is performed
- Program is terminated.